Integer Factorization as an Ising problem

Anil Prabhakar, Advith Desu, Aryan Namboodri

Indian Institute of Technology Madras

May 11, 2026

Factorization Problem

- Given a natural number N, find P, Q such that

$$N = PQ$$

- Hard inverse problem underlying RSA security.
- Map factorization $\rightarrow$ QUBO (or Ising) problem.
- Solve using annealing-based algorithms.

Binary Representation and Assumptions

Assumptions:

- N is odd.
- Factors have equal bit-length

$$n = \left\lceil \tfrac{1}{2} \log_2 N \right\rceil .$$

Binary form:

$$P = \sum_{k=0}^{n-1} 2^k p_k, \quad Q = \sum_{k=0}^{n-1} 2^k q_k, \quad p_k, q_k \in \{0, 1\}.$$

Fixed bits:

$$p_0 = q_0 = 1, \qquad p_{n-1} = q_{n-1} = 1.$$

Column-wise Multiplication

- Construct binary long multiplication table.
- Introduce carry variables $s_{i,j}$.
- Each column gives a constraint:

$$C_i = N_i - \sum_j q_j p_{i-j} - \sum s_{j,i} + \sum 2^j s_{i,i+j}.$$

- Correct factors satisfy $C_i = 0$.

Energy Function

Optimization objective:

$$\min_{\{p_i, q_i, s_{ij}\}} \sum_i (C_i)^2.$$

- Binary variables: factor bits + carry bits.
- Ground state corresponds to valid factorization.
- Alternative direct method:

$$\left(N - \sum p_i 2^i \sum q_j 2^j\right)^2$$

but gives fewer preprocessing opportunities.

Rule 1: Saturated Sum

For binary $x_i \in \{0, 1\}$:

$$\sum_{i=1}^{n} x_i = n \quad \Longrightarrow \quad x_i = 1 \ \forall i.$$

Why: the only way n binary variables sum to n is for every one of them to be 1.

A symmetric form with negative coefficients:

$$\sum_{i=1}^{n} -x_i = -n \quad \Longrightarrow \quad x_i = 1 \ \forall i.$$

Rule 2: Zero Sum

For binary $x_i \in \{0, 1\}$:

$$\sum_{i=1}^{n} x_i = 0 \quad \Longrightarrow \quad x_i = 0 \ \forall i.$$

Why: a sum of non-negative binaries equals 0 only when every term is 0.

The same conclusion holds with negative coefficients:

$$\sum_{i=1}^{n} -x_i = 0 \quad \Longrightarrow \quad x_i = 0 \ \forall i.$$

Rule 3: Doubling Rule

For binary variables $x_1, x_2, x_3 \in \{0, 1\}$:

$$x_1 + x_2 = 2x_3 \quad \Longrightarrow \quad x_1 = x_2 = x_3.$$

Why: LHS $\in \{0, 1, 2\}$ and RHS $\in \{0, 2\}$. Equality forces LHS even, so $x_1 + x_2 \in \{0, 2\}$ — both equal, and equal to x_3.

Use in factorization: appears whenever a column clause has only two factor-bit terms balanced against a single carry, e.g.

$$p_k + q_k = 2s \;\Rightarrow\; p_k = q_k = s.$$

Rule 4: Coefficient Bound Rule

General linear constraint on $x_i \in \{0, 1\}$:

$$\sum_i c_i x_i + C = 0, \qquad S_+ = \sum_{c_i > 0} c_i, \quad S_- = \sum_{c_j < 0} |c_j|.$$

If a single coefficient is larger than the max possible opposing sum, the variable is *forced*:

Positive coefficients ($c_i > 0$):

$$c_i > S_- - C \;\Rightarrow\; x_i = 0, \qquad c_i > S_+ + C \;\Rightarrow\; x_i = 1.$$

Negative coefficients ($c_j < 0$, $|c_j|$ magnitude):

$$|c_j| > S_+ + C \;\Rightarrow\; x_j = 0, \qquad |c_j| > S_- - C \;\Rightarrow\; x_j = 1.$$

Example: $x_1 + x_2 = 2x_3 + 1$. Coefficient of x_3 is 2, max LHS is 2, so x_3 cannot be 1 $\Rightarrow x_3 = 0$.

Rule 5: Complement Rule

For binary $x_1, x_2 \in \{0, 1\}$:

$$x_1 + x_2 = 1 \quad \Longrightarrow \quad x_2 = 1 - x_1, \quad x_1 x_2 = 0.$$

Why: the only solutions are $(0, 1)$ and $(1, 0)$ — exactly one variable is 1, so their product is always 0.

Use in factorization: eliminates one variable outright (substitute $x_2 \to 1 - x_1$) and kills any quadratic term $x_1 x_2$ that appears in a column clause.

Parity Rule

Both sides of a clause must have the same parity:

$$x_1 + x_2 + \text{even} = \text{even} \;\Rightarrow\; x_1 = x_2,$$

$$x_1 + x_2 + \text{even} = \text{odd} \;\Rightarrow\; x_1 + x_2 = 1.$$

Example:

$$x_1 + x_2 + 2x_3 = 2y_1 + 4y_2 \;\Rightarrow\; x_1 = x_2,$$

$$x_1 + x_2 + 2x_3 = 2y_1 + 4y_2 + 1 \;\Rightarrow\; x_1 + x_2 = 1.$$

Why it matters: column clauses balance a small sum of factor bits against carries with even coefficients ($2s_{i,i+1},\ 4s_{i,i+2}, \ldots$). The parity of the factor-bit sum is therefore locked by N_i.

Power Rule

For any binary variable $x \in \{0, 1\}$:

$$x^m = x, \quad m = 2, 3, \ldots$$

Why: $0^m = 0$ and $1^m = 1$ for all positive integers m.

Use in factorization: after squaring the energy function $E = \sum_i C_i^2$, every $p_k^2,\ q_k^2,\ s^2$ collapses back to its linear form $p_k,\ q_k,\ s$. This is what makes the squared objective expressible as a (degree-2) QUBO once cubics/quartics are quadrized.

Worked Example: $N = 391 = 17 \times 23$

Setup:

- $391 = 110000111_2$
- **Fixed bits:** $p_0 = q_0 = 1$ (odd), $p_4 = q_4 = 1$ (MSB)
- **Free bits:** p_1, p_2, p_3 and q_1, q_2, q_3 — 6 unknowns

Long multiplication table (cross-product $p_a q_b$ sits in column $a + b$):

	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
P					1	p_3	p_2	p_1	1
Q					1	q_3	q_2	q_1	1
	1	q_3 p_3	q_2 $p_3 q_3$ p_2	q_1 $p_3 q_2$ $p_2 q_3$ p_1	1 $p_3 q_1$ $p_2 q_2$ $p_1 q_3$ 1	p_3 $p_2 q_1$ $p_1 q_2$ q_3	p_2 $p_1 q_1$ q_2	p_1 q_1	1
Carry-ins	$s_{6,8}, s_{7,8}$	$s_{5,7}, s_{6,7}$	$s_{4,6}, s_{5,6}$	$s_{3,5}, s_{4,5}$	$s_{2,4}, s_{3,4}$	$s_{2,3}$	$s_{1,2}$		
Carry-outs		$s_{7,8}$	$s_{6,7}, s_{6,8}$	$s_{5,6}, s_{5,7}$	$s_{4,5}, s_{4,6}$	$s_{3,4}, s_{3,5}$	$s_{2,3}, s_{2,4}$	$s_{1,2}$	
$N = 391$	**1**	**1**	**0**	**0**	**0**	**0**	**1**	**1**	**1**

Carry $s_{i,j}$ = bit of column i's carry destined for column j (coefficient 2^{j-i}). In each C_k, positive-sign carries enter column k (**carry-ins**); negative-sign carries with weights $2, 4, \ldots$ leave column k (**carry-outs**).

Column Clauses: $C_i = 0$

Each column i: (column sum) + (carries in) $= N_i + 2s_{i,i+1} + 4s_{i,i+2}$.

$$
\begin{aligned}
C_0 &: \ 1 - 1 = 0 \quad \checkmark \\
C_1 &: \ p_1 + q_1 - 1 - 2s_{1,2} = 0 \\
C_2 &: \ p_2 + q_2 + p_1 q_1 + s_{1,2} - 1 - 2s_{2,3} - 4s_{2,4} = 0 \\
C_3 &: \ p_3 + q_3 + p_1 q_2 + p_2 q_1 + s_{2,3} - 2s_{3,4} - 4s_{3,5} = 0 \\
C_4 &: \ 2 + p_1 q_3 + p_3 q_1 + p_2 q_2 + s_{2,4} + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
C_5 &: \ p_1 + q_1 + p_2 q_3 + p_3 q_2 + s_{3,5} + s_{4,5} - 2s_{5,6} - 4s_{5,7} = 0 \\
C_6 &: \ p_2 + q_2 + p_3 q_3 + s_{4,6} + s_{5,6} - 2s_{6,7} - 4s_{6,8} = 0 \\
C_7 &: \ p_3 + q_3 + s_{5,7} + s_{6,7} - 1 - 2s_{7,8} = 0 \\
C_8 &: \ 1 + s_{6,8} + s_{7,8} - 1 = 0
\end{aligned}
$$

Applying Reduction Rules

C_8 **(Rule 2, sum $= 0$):** $s_{6,8} + s_{7,8} = 0 \Rightarrow \boxed{s_{6,8} = s_{7,8} = 0}$.

C_2 **(Rule 4, large coefficient):** LHS terms $p_2 + q_2 + p_1 q_1 + s_{1,2} \leq 4$, but coefficient of $s_{2,4}$ is 4. If $s_{2,4}$=1, need LHS ≥ 5. $\boxed{s_{2,4} = 0}$.

C_1 **(Rule 5):** $p_1 + q_1 = 1 + 2s_{1,2}$. Max sum $= 2 < 3$, so $s_{1,2} = 0$:

$$\boxed{s_{1,2} = 0, \quad q_1 = 1 - p_1} \quad \Rightarrow \quad p_1 q_1 = p_1(1 - p_1) = 0$$

C_2 **(after substitutions):** $p_2 + q_2 = 1 + 2s_{2,3}$. Same logic as C_1:

$$\boxed{s_{2,3} = 0, \quad q_2 = 1 - p_2}$$

C_7 **(Rule 2, sum $= 1$ exactly-one):**

$$p_3 + q_3 + s_{5,7} + s_{6,7} = 1 \quad (s_{7,8} = 0 \text{ already})$$

Exactly one of these four binaries equals 1.

Remaining Structure After Pre-processing

After the rules above, we have eliminated 5 variables ($s_{1,2}, s_{2,3}, s_{2,4}, s_{6,8}, s_{7,8}$) and reduced q_1, q_2 to functions of p_1, p_2.

Free QUBO variables (13):

$\{p_1, p_2, p_3, q_3\} \cup \{s_{3,4}, s_{3,5}, s_{4,5}, s_{4,6}, s_{5,6}, s_{5,7}, s_{6,7}\}$ (plus 2 auxiliaries from quadrization)

Remaining clauses (after substituting $q_1 = 1 - p_1$, $q_2 = 1 - p_2$):

$$
\begin{aligned}
C_3: \;& p_3 + q_3 + p_1(1-p_2) + p_2(1-p_1) - 2s_{3,4} - 4s_{3,5} = 0 \\
&= p_3 + q_3 + p_1 + p_2 - 2p_1p_2 - 2s_{3,4} - 4s_{3,5} = 0 \\
C_4: \;& 2 + p_1q_3 + p_3(1-p_1) + p_2(1-p_2) + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
&= 2 + p_1q_3 + p_3 - p_1p_3 + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
C_5: \;& 1 + p_2q_3 + p_3(1-p_2) + s_{3,5} + s_{4,5} - 2s_{5,6} - 4s_{5,7} = 0 \\
C_6: \;& 1 + p_3q_3 + s_{4,6} + s_{5,6} - 2s_{6,7} = 0 \\
C_7: \;& p_3 + q_3 + s_{5,7} + s_{6,7} = 1
\end{aligned}
$$

Cubic terms appear after squaring (e.g. $p_1p_2 \cdot s_{3,4}$ in C_3^2, $p_1p_3 \cdot s_{4,5}$ in C_4^2) $\Rightarrow$ **quadrization required**.

Quadrization and QUBO Construction

Energy function: $E = \sum_{i=3}^{7} C_i^2$. Squaring introduces cubics like $a \cdot p_i p_j s_{k,\ell}$.

Rosenberg substitution: replace each AB pair appearing in a cubic by an auxiliary w, with penalty

$$P(A, B, w) = M\big(AB - 2Aw - 2Bw + 3w\big), \qquad M \gg \max |\text{coeff}|.$$

$P = 0$ iff $w = AB$. After substitution every cubic $AB \cdot s$ becomes $w \cdot s$ (quadratic).

Auxiliaries needed: $w_1 = p_1 p_2$, $w_2 = p_1 p_3$.

Full QUBO:

$$H = \sum_{i=3}^{7} C_i^2 \Big|_{w_1 \to p_1 p_2,\, w_2 \to p_1 p_3} + M\big[P(p_1, p_2, w_1) + P(p_1, p_3, w_2)\big]$$

Active QUBO variables (13 total):

Original (4)	$p_1,\ p_2,\ p_3,\ q_3$
Carry (7)	$s_{3,4},\ s_{3,5},\ s_{4,5},\ s_{4,6},\ s_{5,6},\ s_{5,7},\ s_{6,7}$
Auxiliary (2)	$w_1 = p_1 p_2,\ w_2 = p_1 p_3$

QUBO Matrix Structure

Sketch of nonzero entries of Q for $N = 391$ (after pre-processing and quadrization):

	p_1	p_2	p_3	q_3	$s_{3,4}$	$s_{3,5}$	$s_{4,5}$	$s_{4,6}$	$s_{5,6}$	$s_{5,7}$	$s_{6,7}$	w_1	w_2
p_1	●	●	●	●								●	●
p_2		●	●	●	●	●						●	
p_3			●	●	●		●	●			●		●
q_3				●			●	●	●	●	●		
$s_{3,4}$					●	●	●	●					
$s_{3,5}$						●	●		●	●			
$s_{4,5}$							●	●	●	●			
$s_{4,6}$								●	●		●		
$s_{5,6}$									●	●	●		
$s_{5,7}$										●	●		
$s_{6,7}$											●		
w_1												●	
w_2													●

The ground state of this QUBO recovers P=17, Q=23 with E=0.

QUBO $\rightarrow$ Ising Conversion

QUBO uses $x_i \in \{0, 1\}$; Ising-form annealers (D-Wave, our GPU SA) use spins $\sigma_i \in \{-1, +1\}$.

Linear change of variables:

$$\sigma_i = 1 - 2x_i \qquad \Longleftrightarrow \qquad x_i = \tfrac{1-\sigma_i}{2}.$$

Substitute into the QUBO $H = \sum_i Q_{ii}\, x_i + \sum_{i<j} Q_{ij}\, x_i x_j$ and collect terms:

$$H(\boldsymbol{\sigma}) = \underbrace{\sum_i h_i \sigma_i}_{\text{linear field}} + \underbrace{\sum_{i<j} J_{ij}\, \sigma_i \sigma_j}_{\text{couplings}} + \text{const.}$$

Coefficients (treating Q symmetrically so $Q_{ij} = Q_{ji}$ for $i \neq j$):

$$\boxed{J_{ij} = \tfrac{1}{4} Q_{ij}}, \qquad \boxed{h_i = -\tfrac{1}{2} Q_{ii} - \tfrac{1}{4} \sum_{j \neq i} Q_{ij}}.$$

J matrix Visualization I

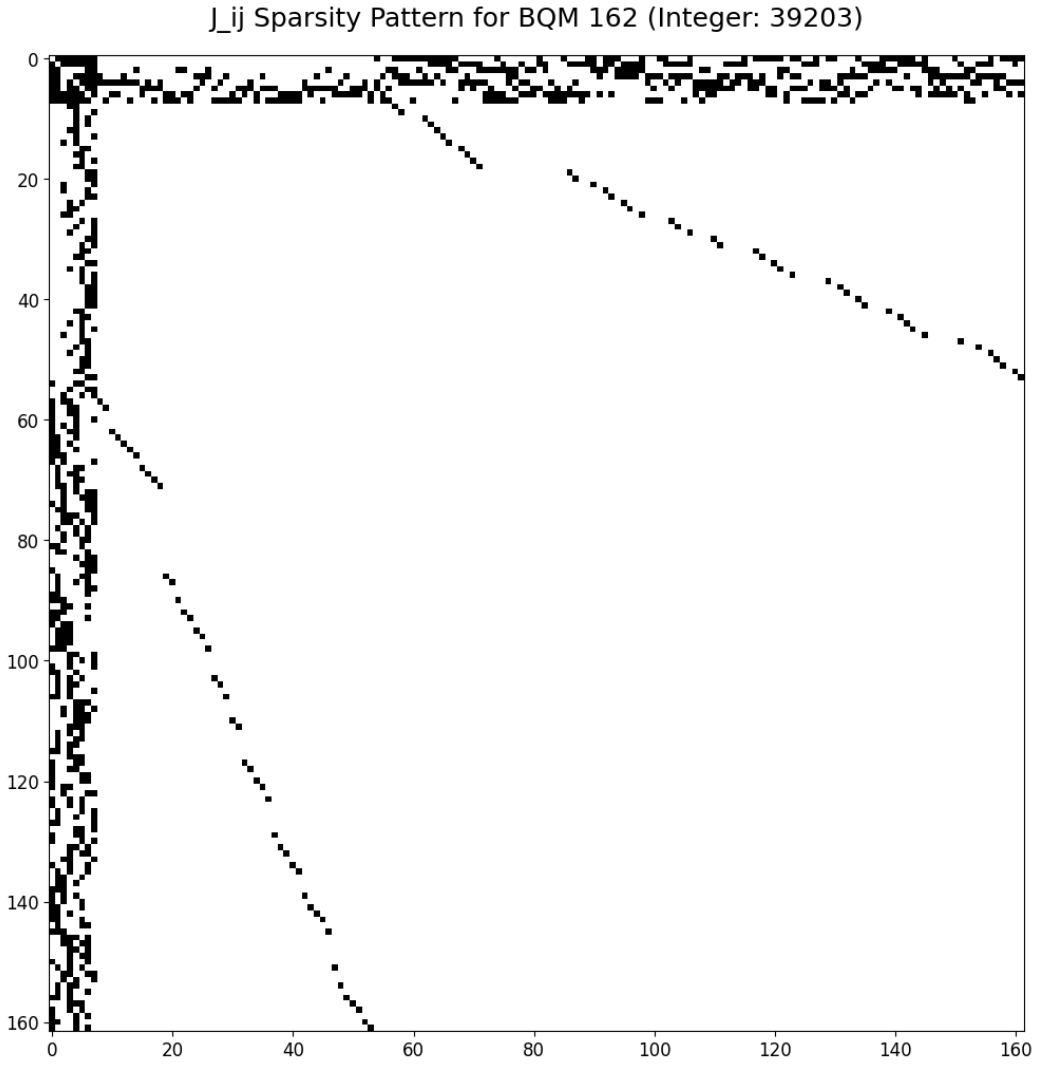


Figure: Sparsity of J matrix for 162 spins (Integer 39203).

J matrix Visualization II

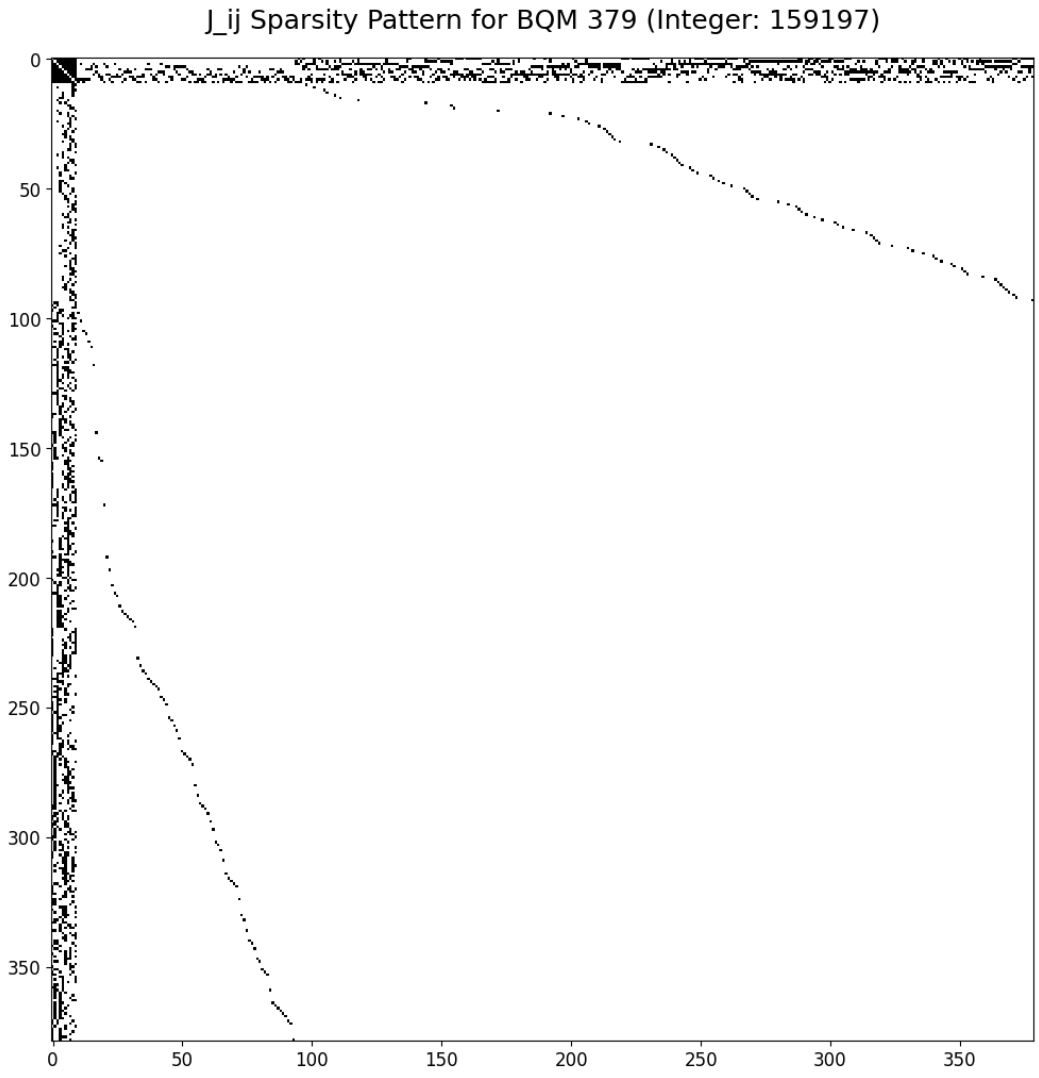


Figure: Sparsity of J matrix for 379 spins (Integer 159197).

J matrix Visualization III

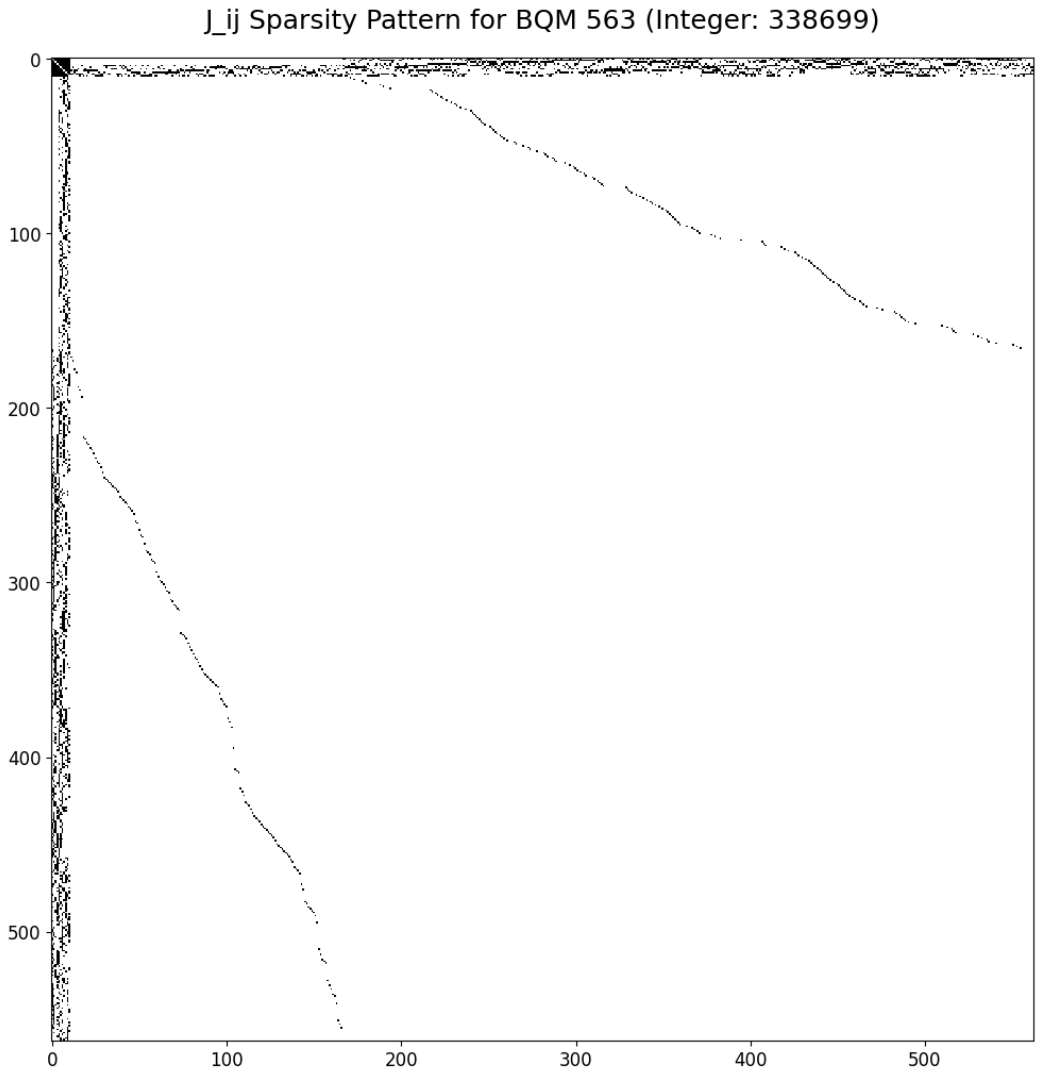


Figure: Sparsity of J matrix for 563 spins (Integer 338699).

Sparse Coupling Matrix Representation (CSR)

Problem: The Ising coupling matrix J_{ij} is very large but mostly zero. Storing it as a dense $N \times N$ matrix wastes memory and bandwidth.

Key idea: Store *only the nonzero couplings* using Compressed Sparse Row (CSR) format. CSR represents a sparse matrix using three arrays:

$$\texttt{row\_ptr}[i] \quad \text{start index of row } i$$

$$\texttt{col\_idx}[k] \quad \text{column index of nonzero entry}$$

$$\texttt{values}[k] \quad \text{value of } J_{ij}$$

Only nonzero entries are stored.

Memory cost: $\mathcal{O}(N + \text{nnz})$ instead of $\mathcal{O}(N^2)$ (nnz = number of nonzeros).

CSR Example

Consider a symmetric 5×5 coupling matrix with 8 non-zero entries (out of 25):

$$J = \begin{pmatrix} \cdot & 2 & \cdot & \cdot & -1 \\ 2 & \cdot & \cdot & 3 & \cdot \\ \cdot & \cdot & \cdot & \cdot & 4 \\ \cdot & 3 & \cdot & \cdot & \cdot \\ -1 & \cdot & 4 & \cdot & \cdot \end{pmatrix}$$

($\cdot$ denotes a stored zero — not represented in CSR)

CSR representation:

index	0	1	2	3	4	5	6	7
`col_idx`	1	4	0	3	4	1	0	2
`values`	2	-1	2	3	4	3	-1	4

index	0	1	2	3	4	5
`row_ptr`[i]	0	2	4	5	6	8

Reading row i: entries in row_ptr[i] represent the start of the i^{th} row in the J values array. Entries are stored at indices $\texttt{row\_ptr}[i] \ldots \texttt{row\_ptr}[i{+}1]{-}1$ of the column index.

Memory cost: $\mathcal{O}(N + \text{nnz})$ instead of $\mathcal{O}(N^2)$ (nnz = number of nonzeros).

GPU Kernel

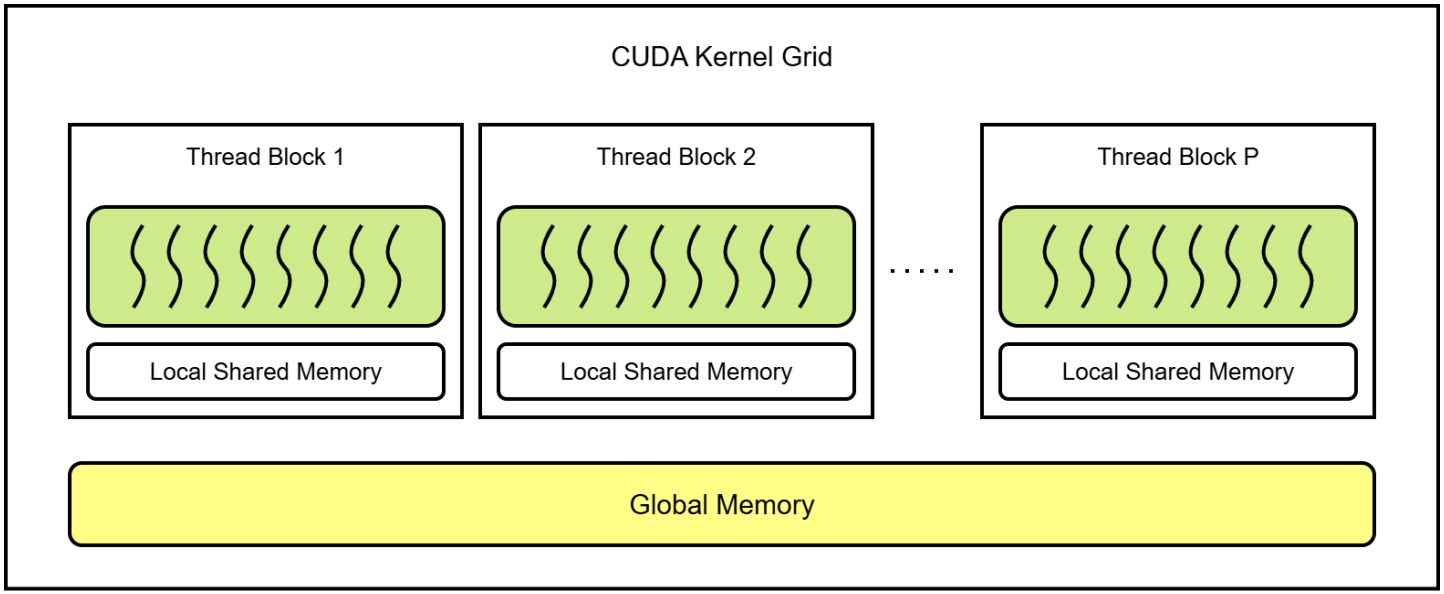


Figure: GPU Threads and Thread blocks.

GPU Thread Calculation in DA1

Each thread i calculates the local field $L_i = \sum_j J_{ij} s_j + h_i$ and the energy difference is then calculated as $-2 \times s_i \times L_i$ depending on which i^{th} spin flipped.

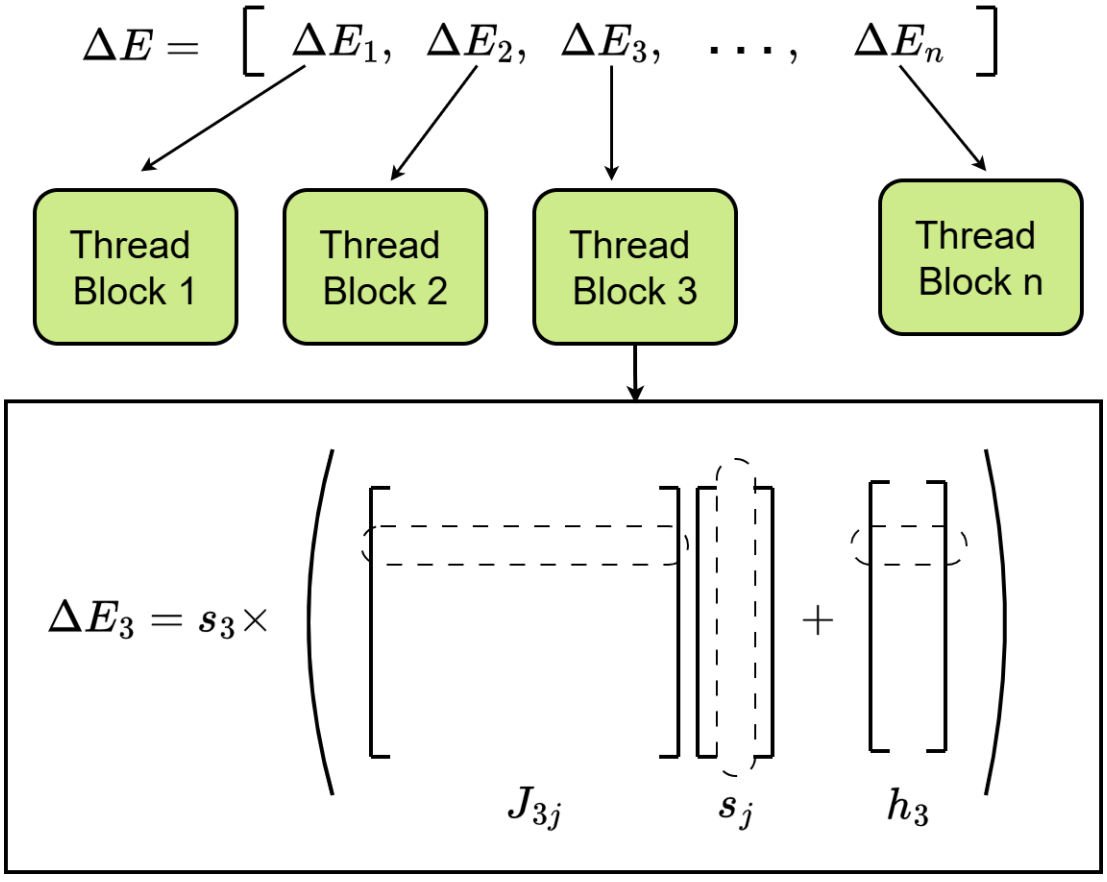


Figure: Energy calculation in DA1.

Graph-Coloring Simulated Annealing

In one Metropolis sweep we want to flip many spins in parallel, but two spins coupled by a non-zero J_{ij} share an edge and cannot be flipped simultaneously without invalidating each other's ΔE.

We use graph coloring on the interaction graph (nodes = spins, edges = nonzero J_{ij}) partitions spins into colors such that *no two same-color spins are coupled*. All spins in one color can then be flipped in parallel without affecting each other's energy calculations.

Two performance bins based on row degree $d_i = |\{j : J_{ij} \neq 0\}|$:

- **Dense** ($d_i \geq 128$): full block of 1024 threads, shared-memory reduction.
- **Sparse** ($d_i < 128$): one warp (32 threads) per spin, 32 spins packed per block, warp-shuffle reduction.

Number of Total and Densely Connected Spins with Scale

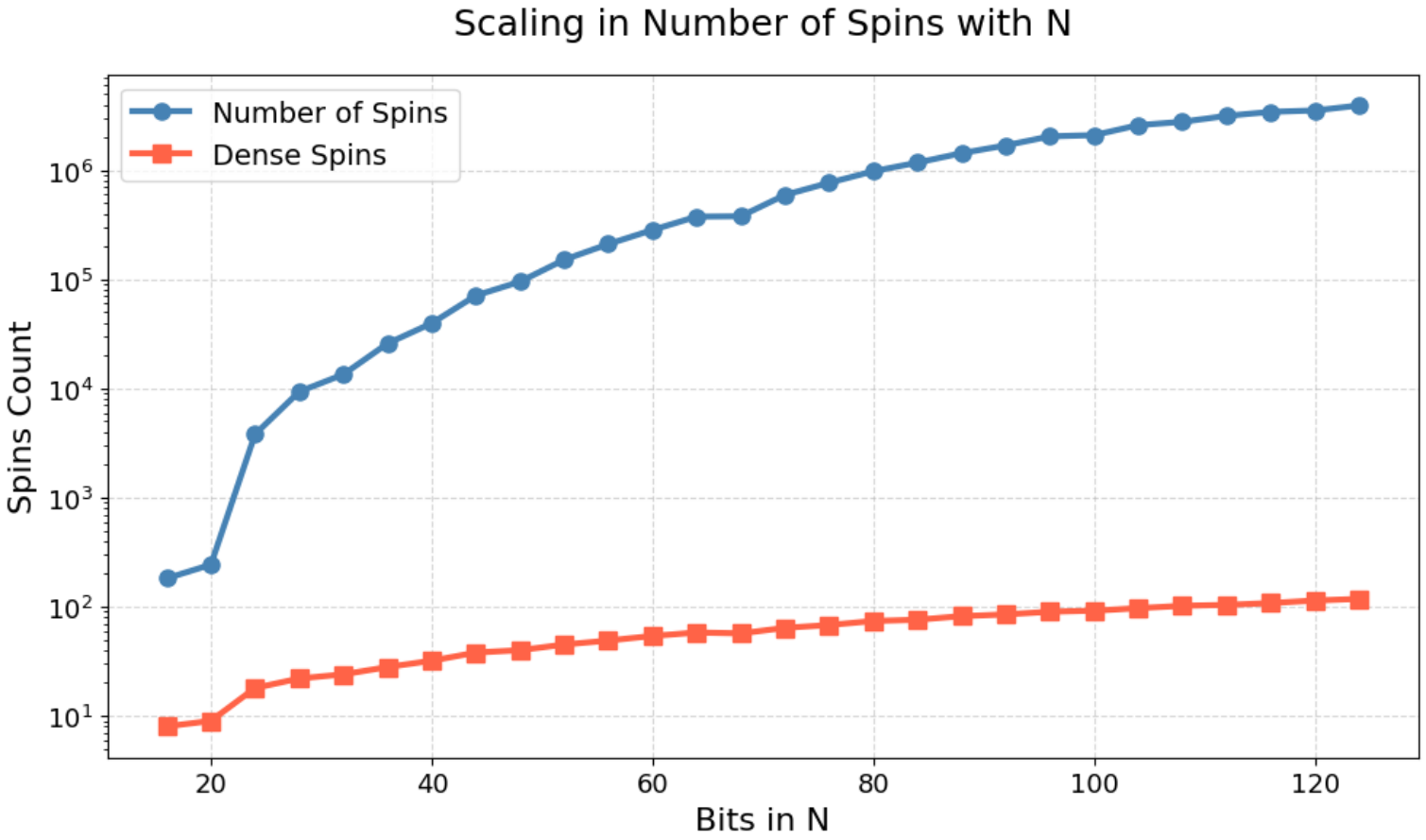


Figure: Number of bits in N vs number of densely connected spins.

One-time Setup

Algorithm Pre-processing steps before annealing

Require: CSR of symmetric J, linear field h, num_spins N

1: Verify J symmetry and a zero diagonal
2: Bin each spin: **dense** if $d_i \geq 128$, else **sparse**
3: color[ ] $\leftarrow$ Welsh–Powell greedy coloring (largest-degree-first, first-fit)
4: Bucket spins by color: `color_dense_offsets`, `color_sparse_offsets`
5: Upload all CSR + color tables + hub arrays to GPU
6: Initialize a random spin configuration and compute initial energy
7: Calculate the start-temperature: $T_0 = \langle \Delta E^+ \rangle / \ln(1/\chi)$
(χ = target acceptance rate)

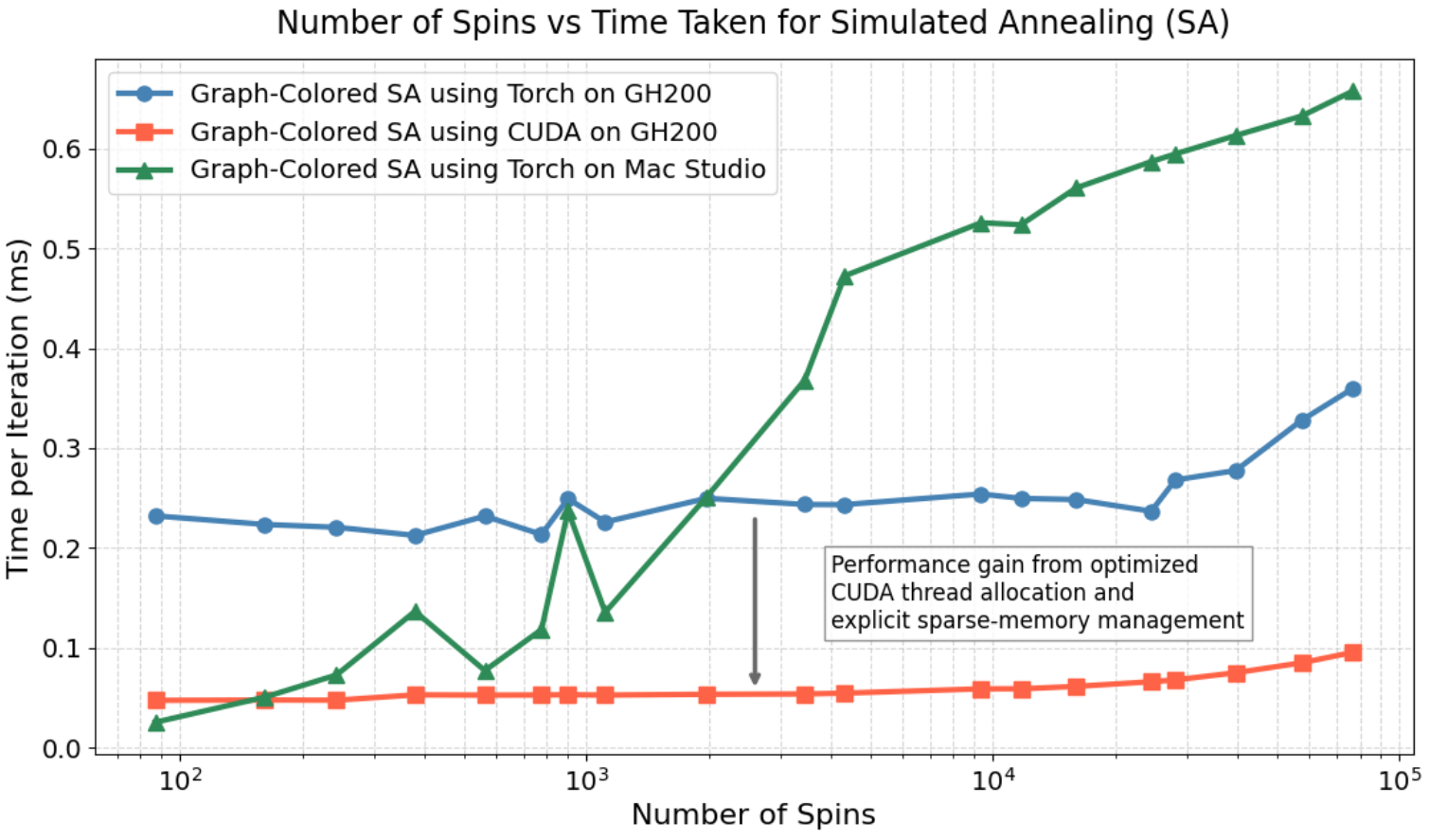
Annealing Loop (GPU)

Algorithm Color-parallel simulated annealing

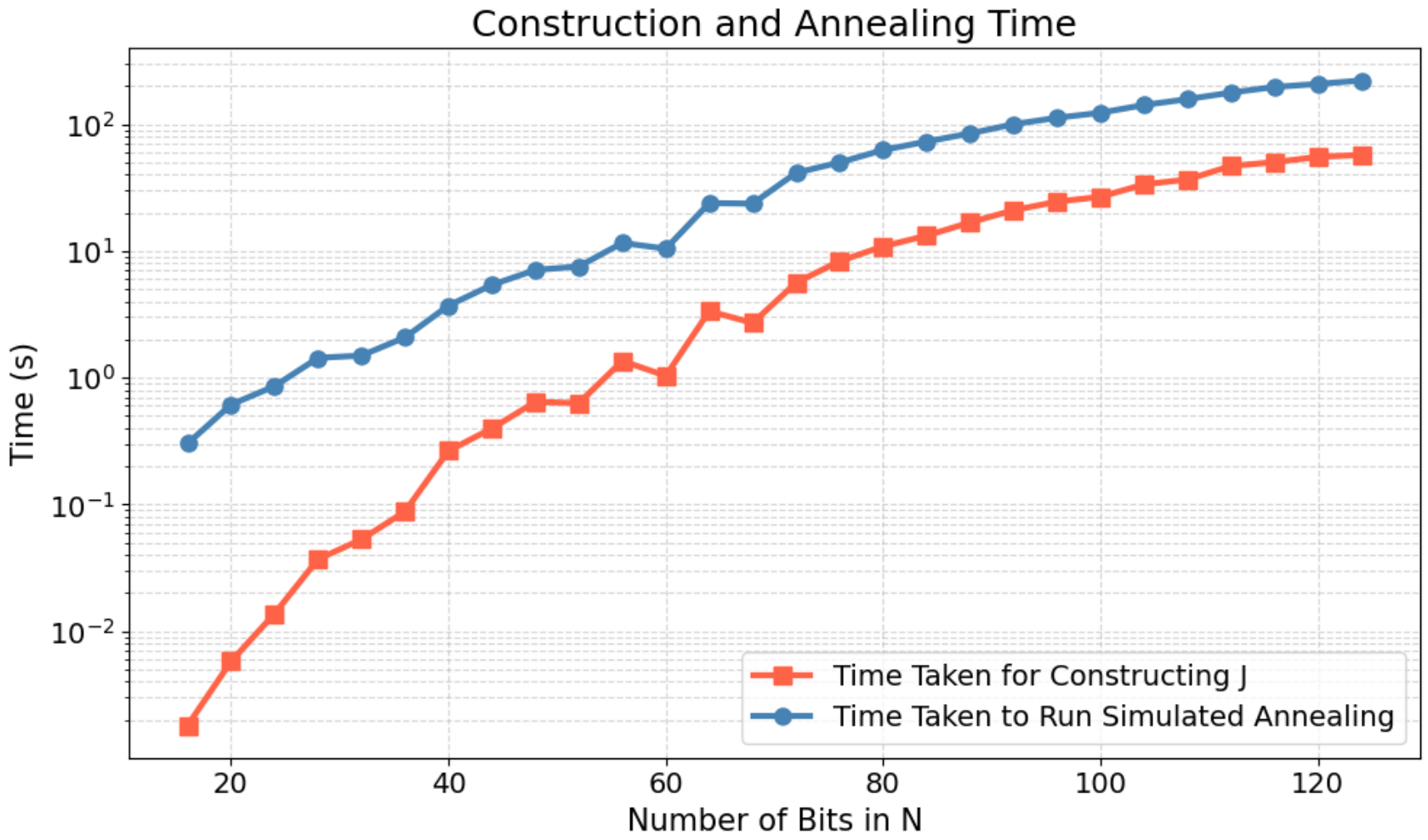
Require: β-schedule $\{\beta_t\}$ (geometric), sweeps per β, N_{colors}

for β in β-schedule **do**
  for sweep $= 1 \ldots m$ **do**
    Generate fresh uniform randoms for all N spins
    for $c = 0 \ldots N_{\text{colors}} - 1$ **do** ▷ colors run sequentially
      collectFlipCandidates⟨dense⟩ on color c
      collectFlipCandidates⟨sparse⟩ on color c
        each spin i in color c **in parallel**:
          $L_i \leftarrow \sum_j J_{ij} s_j + h_i$, $\Delta E_i \leftarrow -2 s_i L_i$
          if $\text{rand}_i < \min(1, e^{-\beta \Delta E_i})$: append $(i, \Delta E_i)$ to candidates
      applyAllFlipsInColor: flip every accepted spin, atomicAdd ΔE_i to total energy
      if any hub spin in color c: refresh hub-values cache
    end for
  end for
  if total energy $<$ best energy **then** snapshot best spins
  end if
end for
return best-energy spin configuration

Annealing Time per Iteration vs Number of Spins

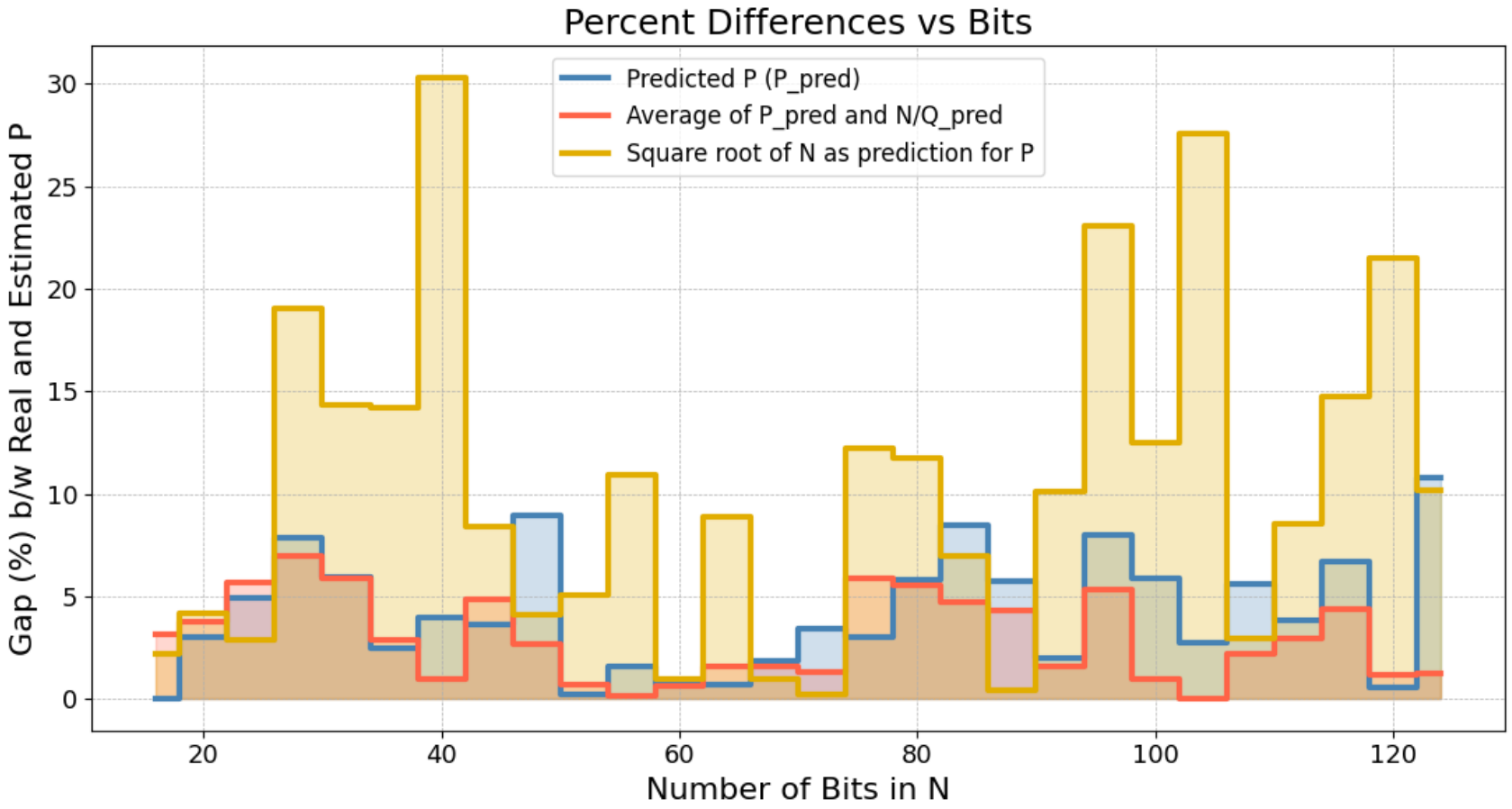


Time Taken for Construction and Annealing



Each Annealing run consists of 10 sweeps at each temperature step, with the starting temperature T_0 calculated to give an initial acceptance rate of 0.5, and a geometric cooling schedule with factor 0.95 until $T < 10^{-3}$.

Solution Quality Measured by Gap between Real and Estimated P



1. Average (%) difference with square root N as the estimate = **10.33%**
2. Average (%) difference with P_{pred} from Simulated Annealing = **4.23%**
3. Average (%) difference with mean of P_{pred} and N/Q_{pred} = **2.98%**

Brute-force Search

The annealer returns approximate factors $P_{\text{pred}}, Q_{\text{pred}}$ — usually off by a few percent. A brute-force search around this prediction finds the exact factor.

Best starting estimate — average of two independent guesses for P:

$$P_{\text{centre}} = \tfrac{1}{2}\left(P_{\text{pred}} + \lfloor N/Q_{\text{pred}} \rfloor\right).$$

Outward spiral: test candidates closest to P_{centre} first, alternating above and below until the exact factor is found.

Candidates are grouped into chunks of C. Even chunks spiral upward (steps up), odd chunks spiral downward (steps down). Threads claim the next chunk via an atomic counter, so the search is partially parallel.

Stop condition: the first thread to find $p \mid N$ sets a shared flag; the rest exit at the next iteration.

Brute-force Optimizations

Testing all numbers in a chunk is ineffecient as we can filter out many candidates without a divisibility test using a wheel sieve cheaply. The wheel sieve eliminates candidates that are divisible by small primes.

Wheel Sieve:

- Any p divisible by $2, 3, 5, 7$, or 11 cannot be the (odd, ≥ 13) prime factor.
- Using a Wheel sieve skips approximately 79% of candidates without a divisibility test.

Early termination: atomic `stop` flag checked at each chunk boundary; all T threads exit within one chunk of the discovery.

Brute-force Algorithm

Algorithm Parallel spiral divisor search

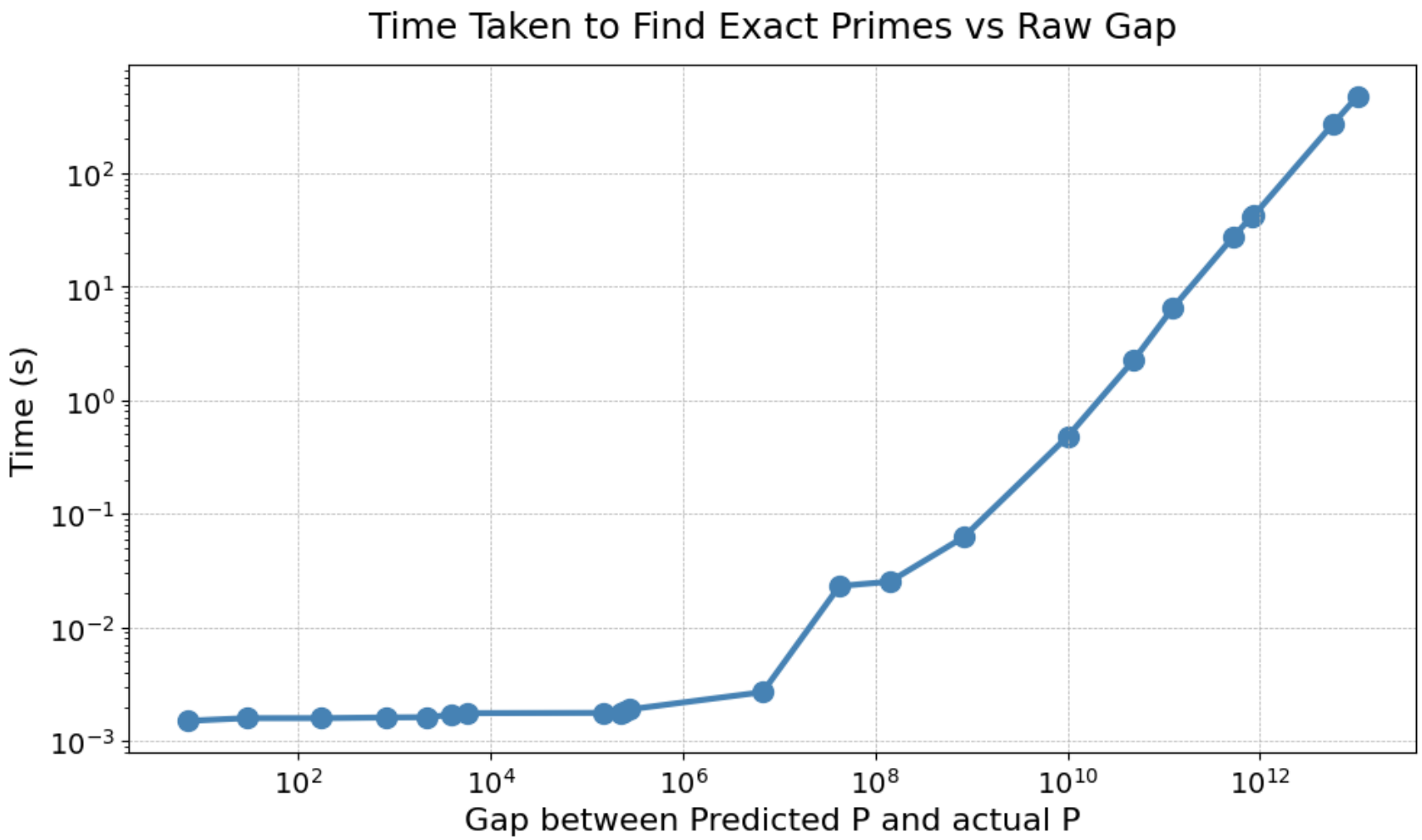
Require: N, P_{pred}, Q_{pred}, T threads

$P_{\text{centre}} \leftarrow \frac{1}{2}(P_{\text{pred}} + \lfloor N/Q_{\text{pred}} \rfloor)$

shared: atomic `chunk_idx` $\leftarrow 0$, atomic `stop` $\leftarrow$ false, atomic `found` $\leftarrow 0$

for each thread **in parallel do**
    while not `stop` **do**
        $k \leftarrow$ `chunk_idx.fetchAdd(1)` ▷ claim next chunk
        if k even **then** ▷ step up
            $[\text{lo}, \text{hi}] \leftarrow [P_{\text{centre}} + \frac{k}{2}C,\ P_{\text{centre}} + (\frac{k}{2}+1)C - 1]$
        else ▷ step down
            $[\text{lo}, \text{hi}] \leftarrow [P_{\text{centre}} - \frac{k+1}{2}C,\ P_{\text{centre}} - \frac{k-1}{2}C - 1]$
        end if
        for each $p \in [\text{lo}, \text{hi}]$ passing the wheel sieve **do**
            if $N \bmod p = 0$ **then**
                `found` $\leftarrow p$; `stop` $\leftarrow$ true; **return**
            end if
        end for
    end while
end for
return $P \leftarrow$ `found`, $Q \leftarrow N/P$

Time Taken to Find Exact primes by Brute Force

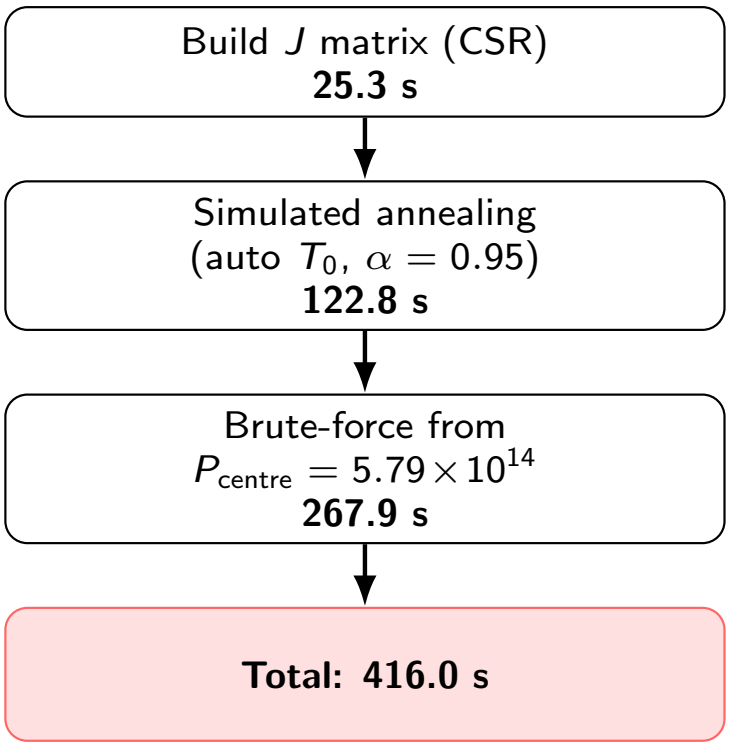


$$\text{Time Taken (s)} = \frac{3.32 \times 10^{-9}}{\text{Threads}} \times \text{Gap (\%)} \times 2^{n}$$

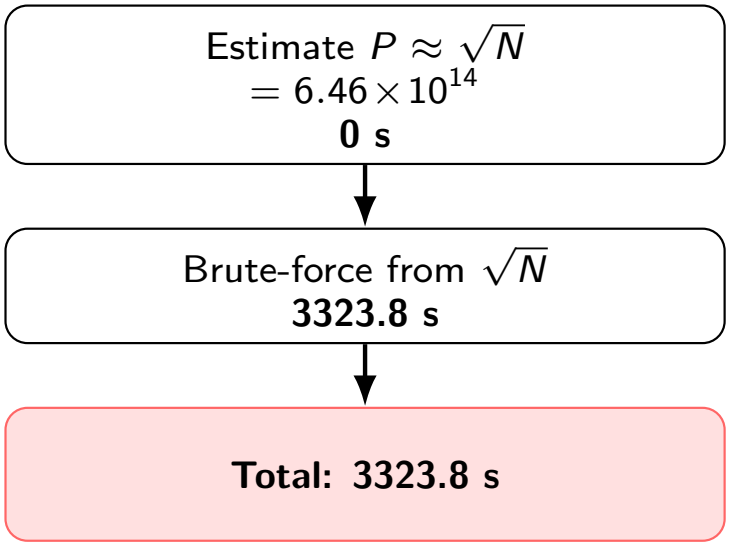
End-to-End Run: ∼100-bit Semiprime

$$N = 416\,749\,328\,744\,899\,275\,664\,046\,210\,629$$

Annealing-guided pipeline



Naive $\sqrt{N}$ baseline



Factors found: $P = 573\,858\,364\,692\,161$, $Q = 726\,223\,323\,360\,389$ (72 threads)

The annealer's prediction places the brute-force search much closer to the true factor.

Future Work

Better classical post-processing. Brute-force trial division is the simplest way to consume the annealer's guess $(P_{\text{pred}}, Q_{\text{pred}})$, but it makes almost no use of the prediction beyond using it as a starting point. Other Existing classical factorization methods may be able to exploit this kind of information far more effectively than a linear scan. Exploring such methods would be a natural next step.

The Bottleneck, from the brute-force timing model

$$\text{Time(s)} = \frac{3.32 \times 10^{-9}}{\text{threads}} \times \text{Gap(\%)} \times 2^n,$$

- **Threads:** This is the amount of 'compute' we can throw at the problem. Would be helpful, but its bounded by hardware.
- **Gap (%):** a multiplicative factor in front of 2^n. Since 2^n grows exponentially with bit-length, *shrinking the gap helps us keep up* with larger N.

The next step would be to improve the solution quality from the annealer by exploring better QUBO encodings, and any other optimizations that drives P_{pred} closer to P and chips away at the Gap(%).

References

B. Wang, F. Hu, H. Yao, and C. Wang, "Prime factorization algorithm based on parameter optimization of Ising model," *Scientific Reports*, vol. 10, no. 1, 2020. DOI: 10.1038/s41598-020-62802-5.

S. Jiang, K. A. Britt, A. J. McCaskey, T. S. Humble, and S. Kais, "Quantum Annealing for Prime Factorization," *Scientific Reports*, vol. 8, no. 1, 2018. DOI: 10.1038/s41598-018-36058-z.

T. Preis, P. Virnau, W. Paul, and J. J. Schneider, "GPU accelerated Monte Carlo simulation of the 2D and 3D Ising model," *Journal of Computational Physics*, vol. 228, no. 12, pp. 4468–4477, 2009. DOI: 10.1016/j.jcp.2009.03.018.